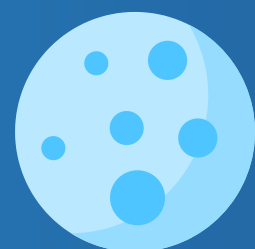




PANDEMIC

Projeto de Software
Parte 3

Equipe



Patricia Carvalho de Moraes
Matheus da Rocha Sena
Italo Ferreira da Silva
Fernando Rene Cabral Araujo
Mateus Sacramento de Matos
Lucas Araujo Sodre



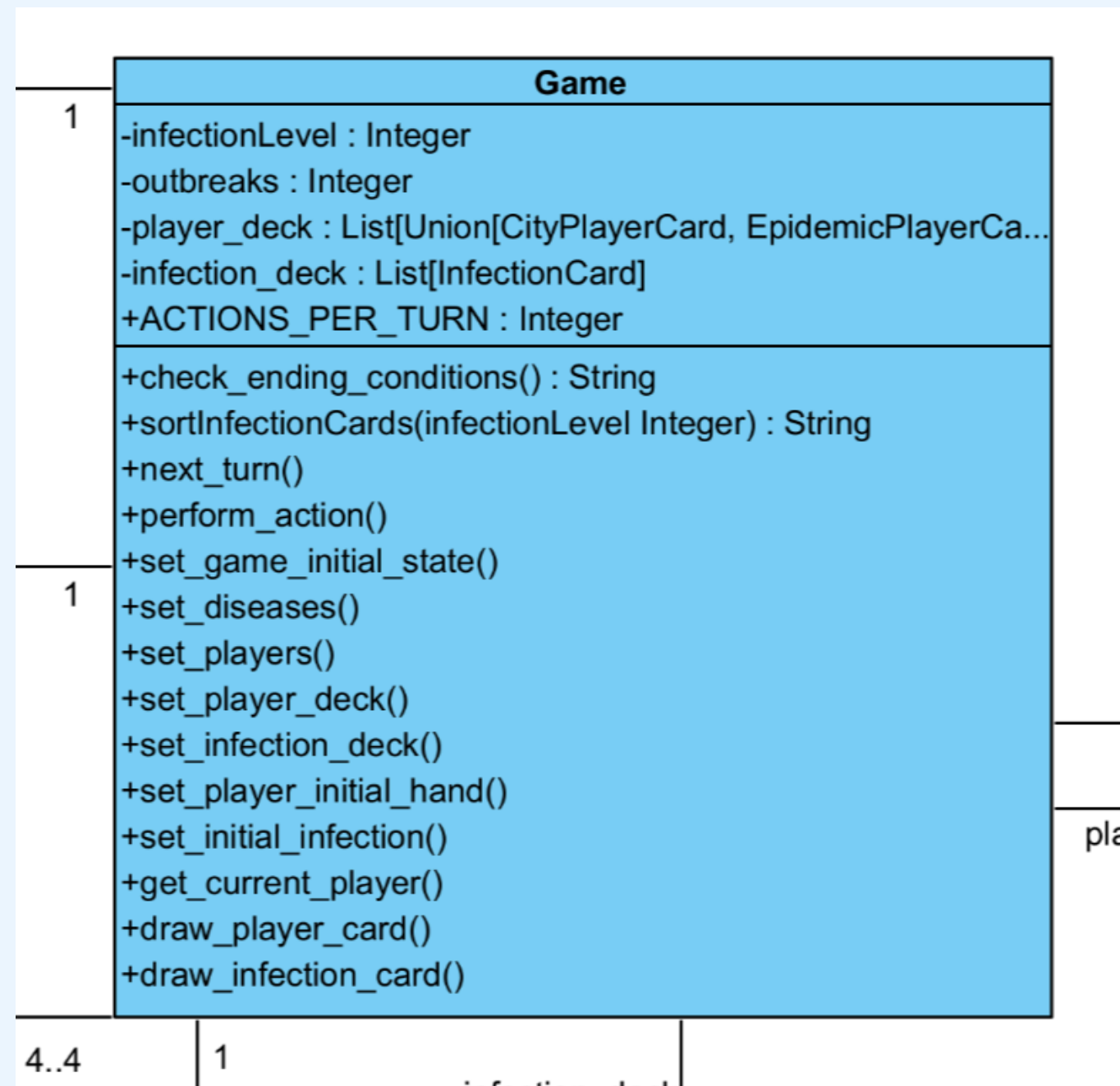
 <https://github.com/Rarkunho/pandemic-uff-gpms-ps.git>  







PADRÃO GRASP



GRASP: Controller (Controlador)

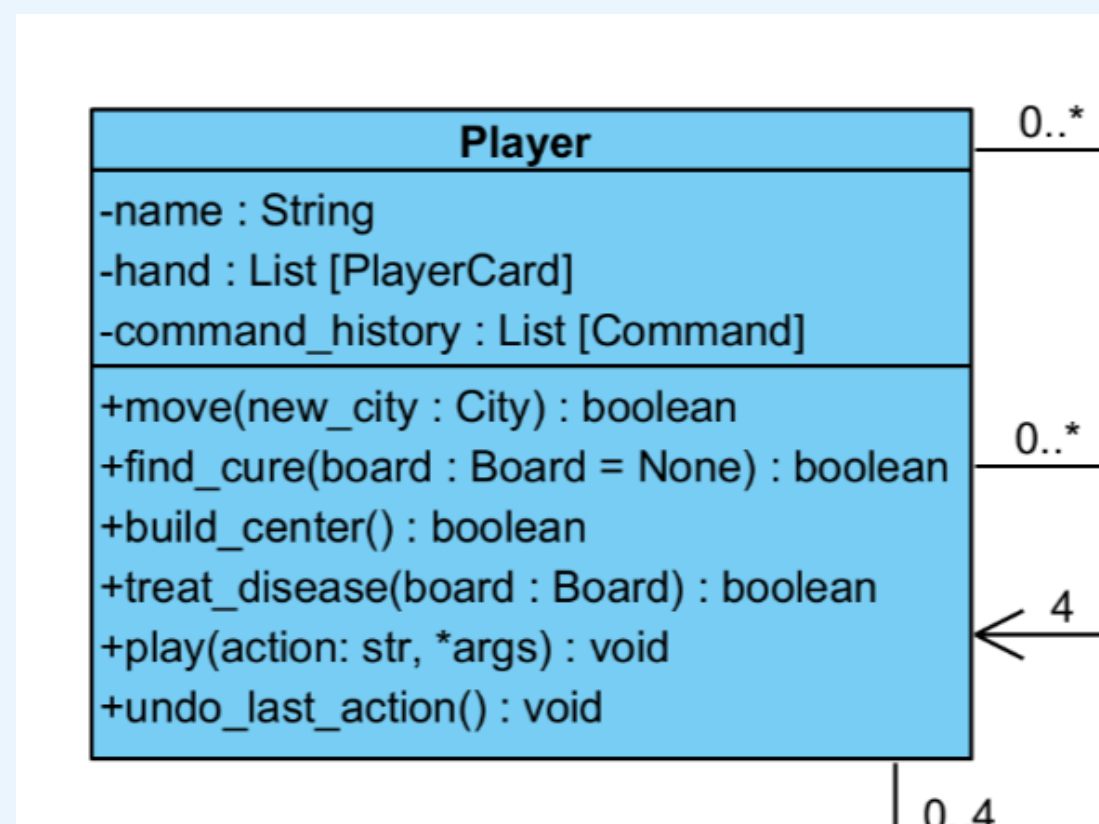
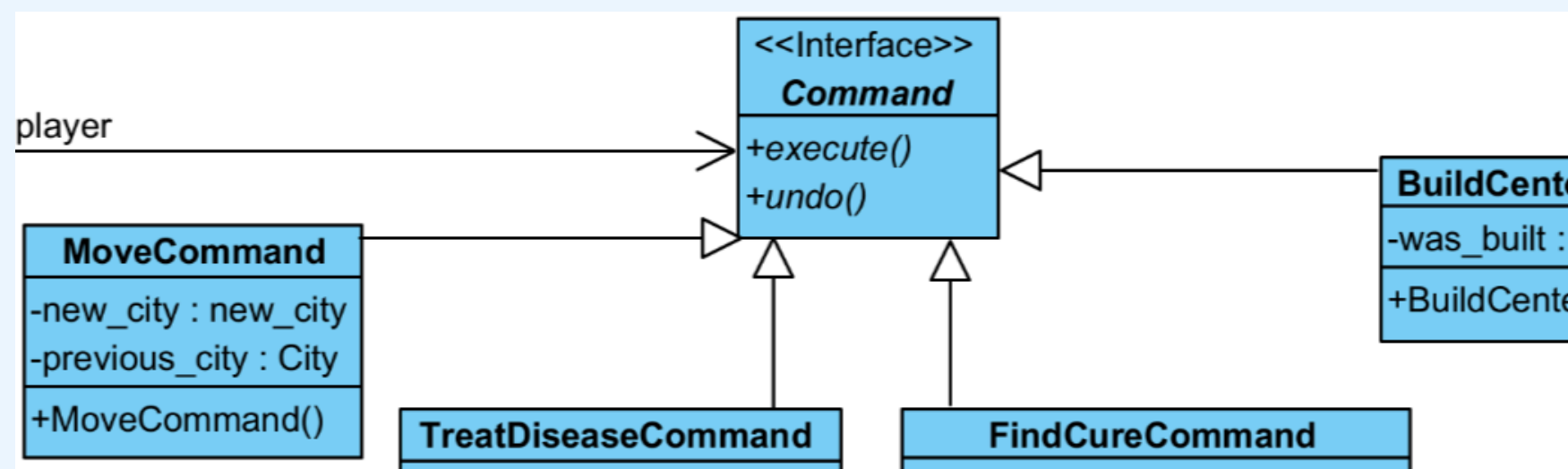
Atribui a responsabilidade de receber e coordenar eventos do sistema a uma classe que represente o sistema como um todo ou um cenário de caso de uso.

A classe *Game* atua como controlador central do sistema, gerencia o estado global do jogo e coordena as interações entre os principais componentes através de métodos como *checkEndingConditions* e *sortInfectionCards*





PADRÃO GRASP



GRASP: Baixo Acoplamento e Alta Coesão

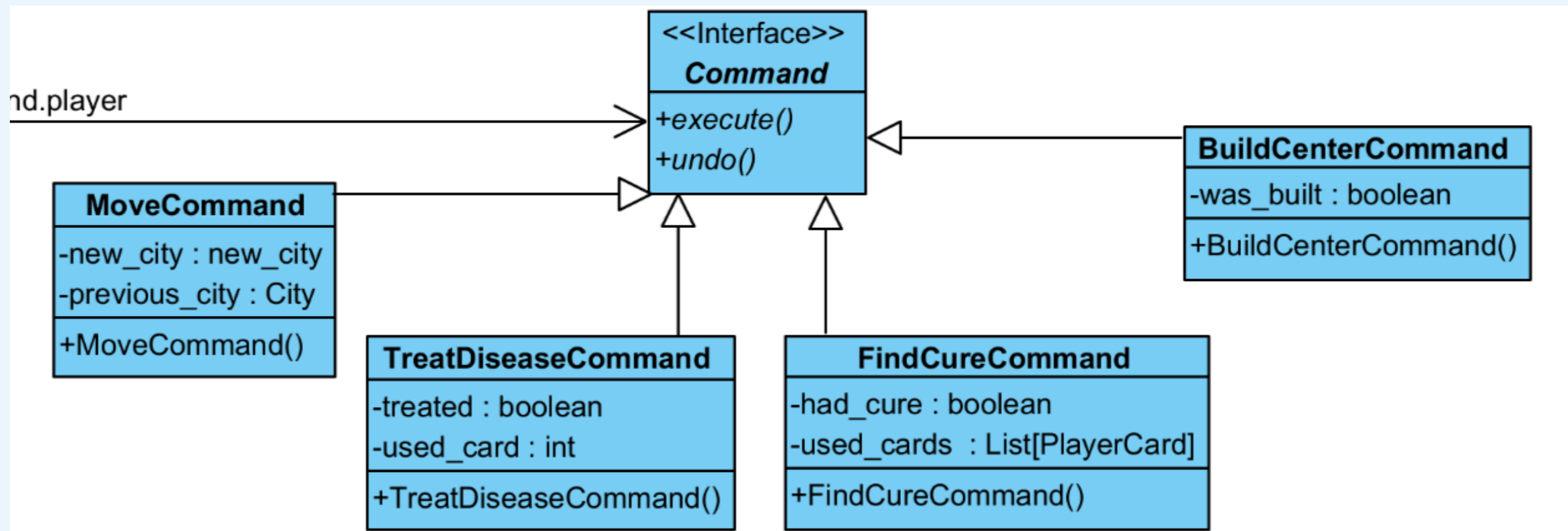
O acoplamento mede o quão fortemente um elemento está conectado a outros elementos, enquanto a coesão é uma medida de quão relacionadas e focadas são as responsabilidades de um elemento.



o uso de interfaces como *Command* promove baixo acoplamento entre quem invoca uma ação e quem a executa. As classes têm responsabilidades bem definidas (alta coesão), como *Player*, focado nas ações do jogador.



PADRÃO GOF



GoF: Command

Encapsula uma solicitação como um objeto, permitindo parametrizar clientes com diferentes solicitações, enfileirar ou registrar solicitações e suportar operações que podem ser desfeitas.

O diagrama mostra a interface **Command** com os métodos `execute()` e `undo()`, além das implementações concretas, como **MoveCommand** e **TreatDiseaseCommand**, permitindo que as ações do jogo sejam encapsuladas como objetos.



DIAGRAMA DE COMUNICAÇÃO

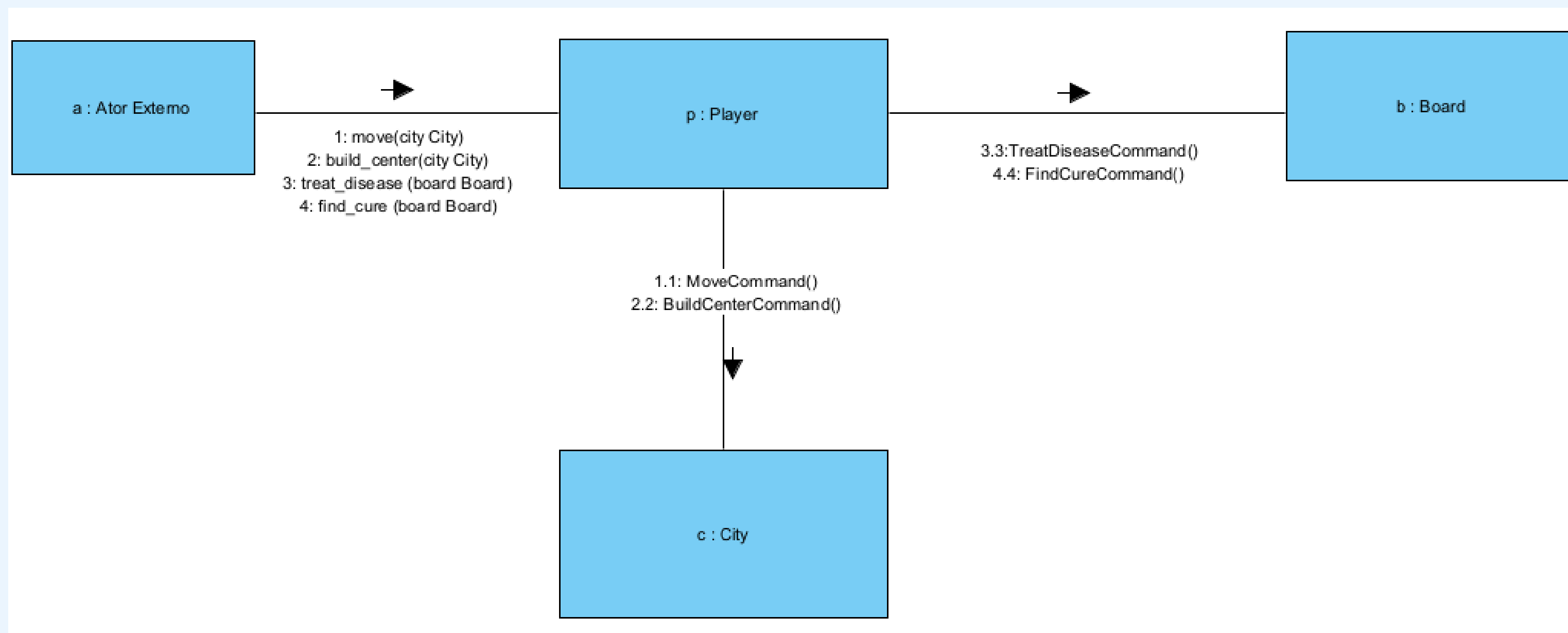
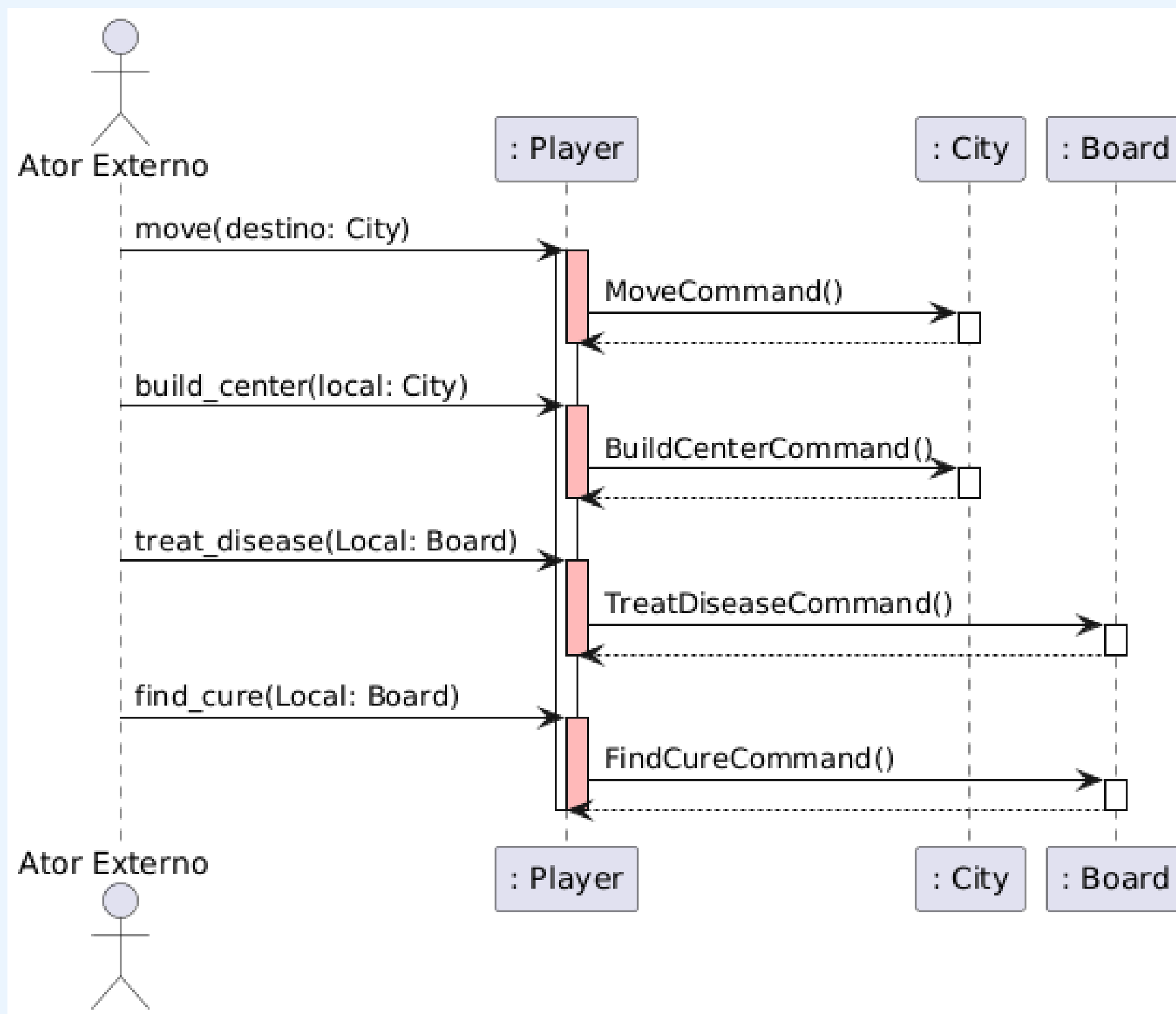




DIAGRAMA DE SEQUÊNCIA





DIFICULDADES

- Diagrama de classes;
- Uso do Visual Paradigm;
- Necessidades no projeto só percebidas durante o desenvolvimento, que tornaram necessário alterar o diagrama de classes (e por consequência, o diagrama de sequência e de comunicação);
- Compreensão das regras do jogo e adaptação delas ao código e diagramas;
- Implementação dos surtos (geram loops);
- Implementação da interface gráfica;





DECISÕES TOMADAS

- Mudança da restrição: "O sistema deve permitir iniciar um jogo com 2 a 4 instâncias de Player". Foi fixada a quantidade de 4 jogadores.
 - Motivo: redução do escopo.
- Adaptação da regra original: "O sistema deve permitir ao Player invocar o método `Player.buildCenter()` na City atual, descartando a `CityPlayerCard` correspondente e definindo `City.hasCenter = true`." Não é necessário descartar a `CityPlayerCard` após essa invocação.
 - Motivo: Construir centros não consome cartas nessa versão do jogo pois as cartas foram simplificadas apenas para tratar doenças e curar surtos





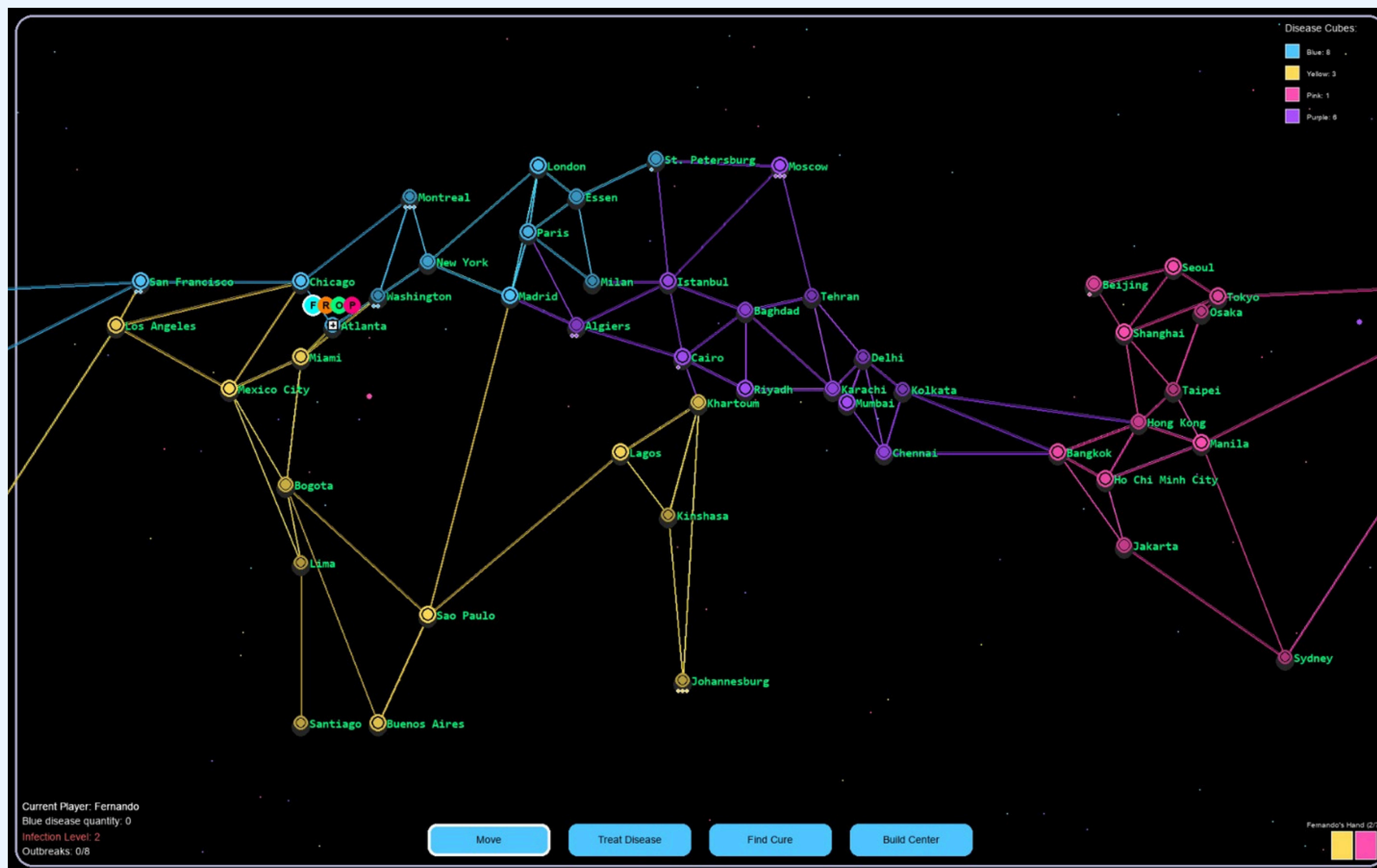
DECISÕES TOMADAS

- Criação da classe Board, que só foi percebida a necessidade para o funcionamento ao final da implementação do jogo.
 - Gerencia partes visuais do jogo e eventos de input do pygame
- Mudança da restrição: “A interface será via Linha de Comando (CLI), sem uso de bibliotecas gráficas complexas”. A interface gráfica foi implementada usando a biblioteca Tkinter.
 - Motivo: melhoria da usabilidade e facilidade de uso, uso de funcionalidades adicionais, adoção de um visual mais amigável para os jogadores;





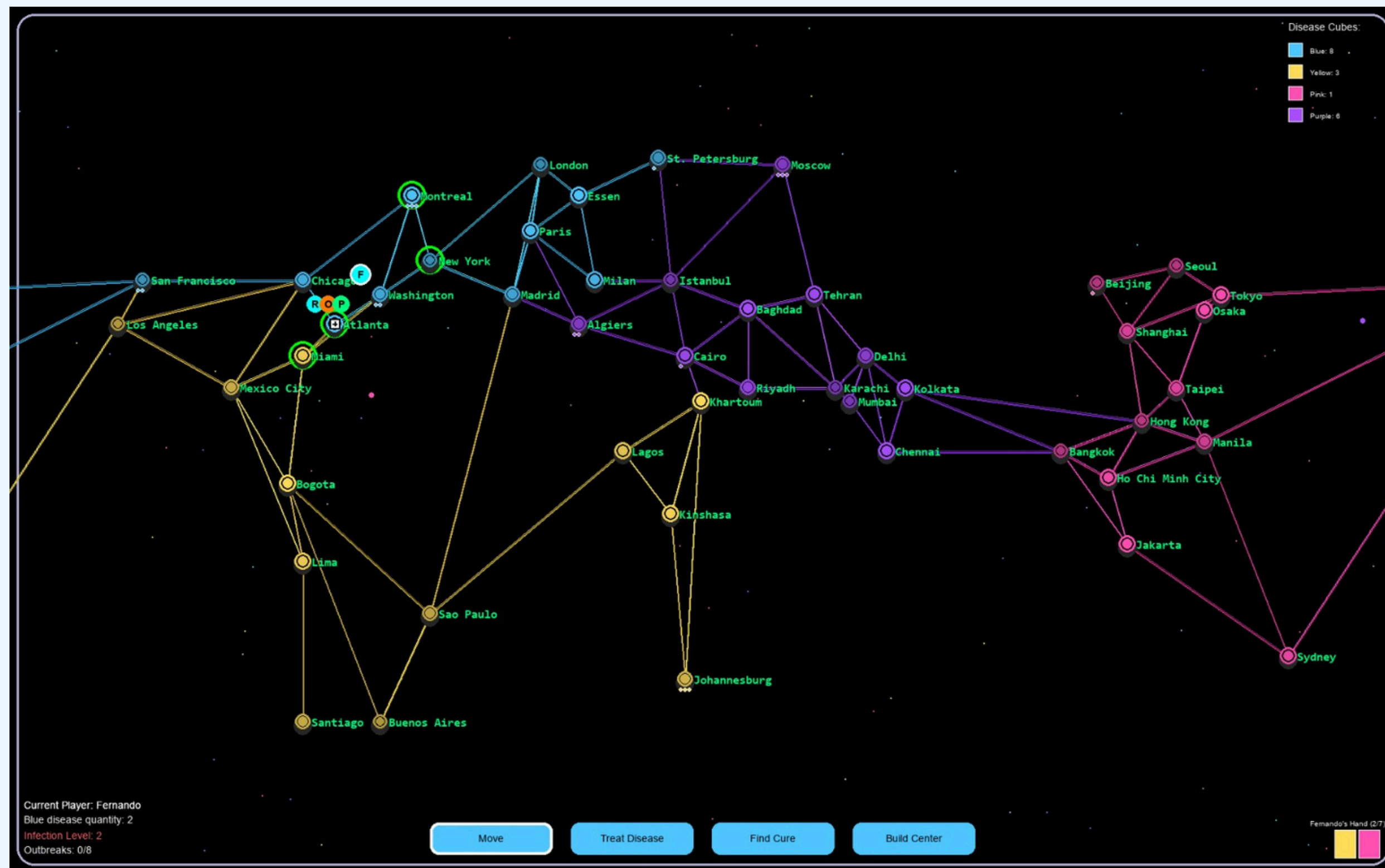
VERSÃO FINAL



Começo do jogo



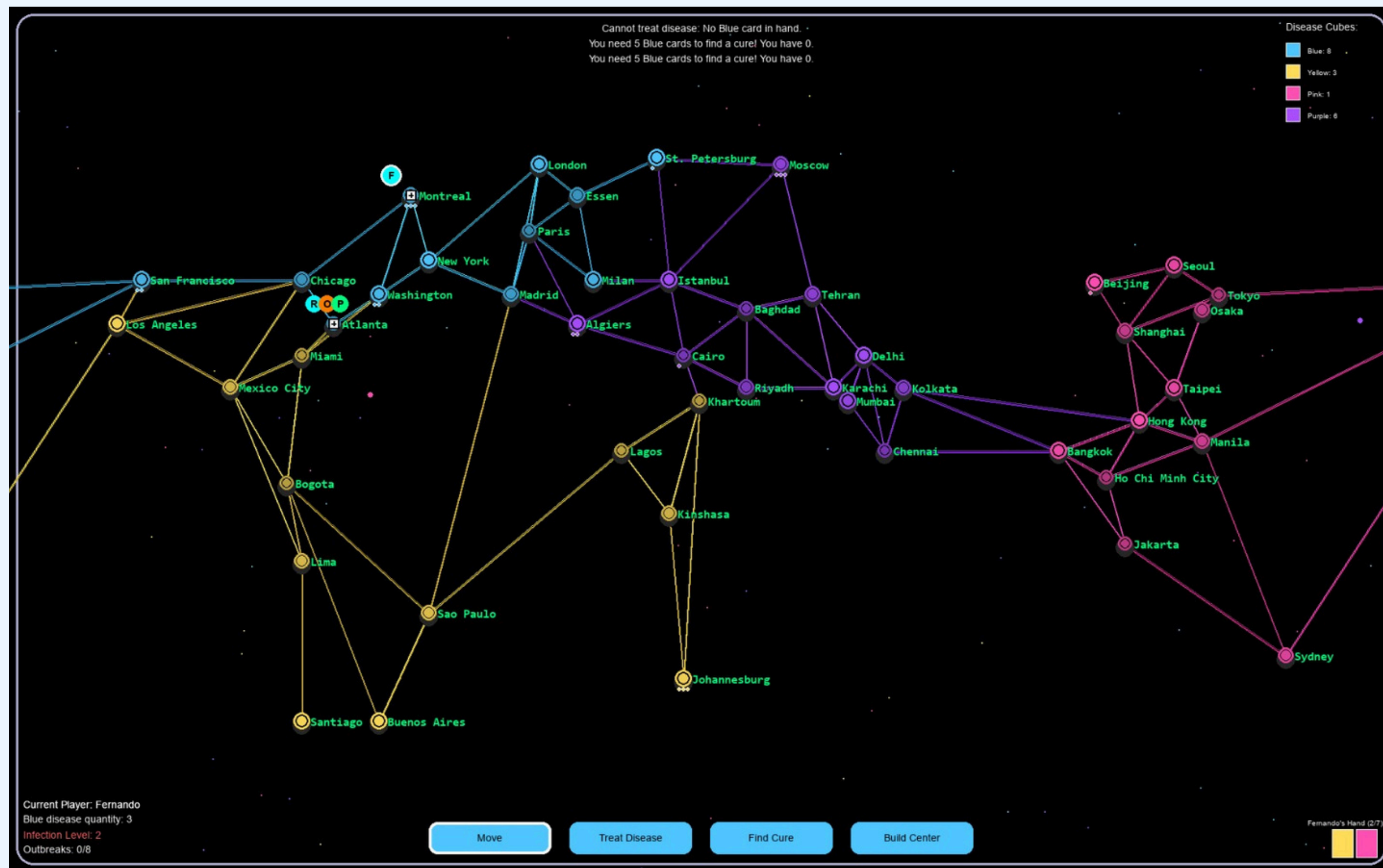
VERSÃO FINAL



Escolhendo o comando de MOVER com o jogador Fernando, mostrando opções



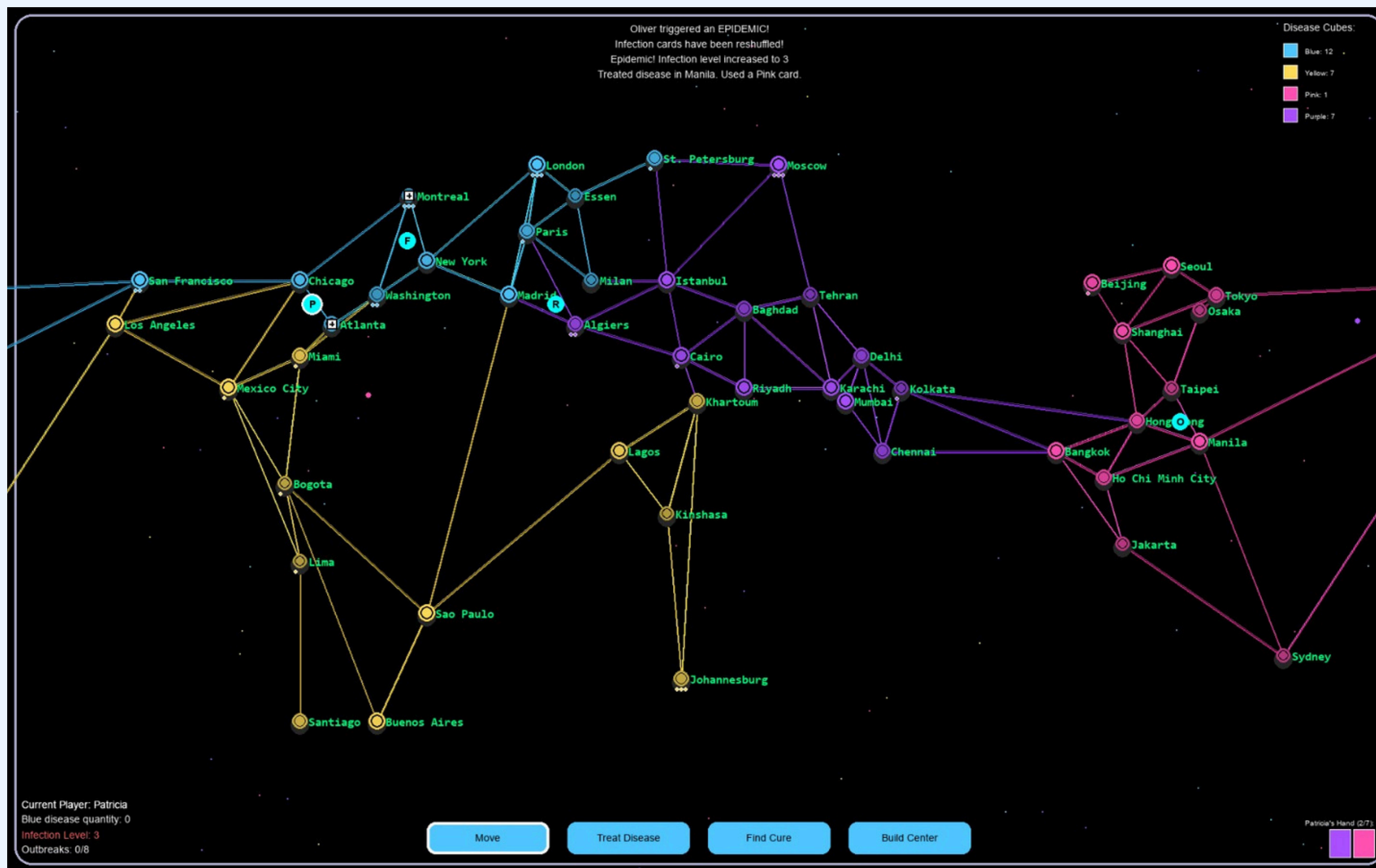
VERSÃO FINAL



Tentando curar doença
e achar cura quando não
tem cartas



VERSÃO FINAL



Jogador oliver trata uma doença em Manila com uma carta Rosa (trocou o turno, as cartas ali embaixo são da Patricia). Surto ocorre por excesso de cubos

OBRIGADO!